ФАХОВИЙ КОЛЕДЖ
МНТУ

Об'єктно орієнтоване програмування

Практичне заняття №5. ЗМ2. ЛЗ6.

Тищенко Сергій, alk-43

2026-03-22

1 Шаблонний клас пошуку елементів у контейнері з використанням ітераторів, предикатів та лямбда виразів у сучасному C++

1.1 Мета

Ознайомитися з можливостями стандартної бібліотеки C++ для узагальненого програмування, з використанням шаблонів, контейнерів, ітераторів, алгоритмів, предикатів та лямбда виразів. Практично реалізовувати універсальні механізми пошуку елементів у колекціях із застосуванням сучасних засобів мови C++.

1.2 Опис завдання

Реалізувати шаблонний клас для виконання послідовного пошуку елементів у контейнері. Клас повинен підтримувати роботу з різними типами даних та забезпечувати:

- пошук першого елемента за точним значенням
- пошук першого елемента за умовою
- пошук усіх елементів, що задовольняють задану умову

Реалізація повинна бути узагальненою та використовувати стандартні алгоритми і механізми сучасного C++.

1.3 Опис реалізації

Клас `Finder<T>` параметризується типом `T`, що дозволяє застосовувати його до будь яких типів даних, які підтримують необхідні операції. В якості внутрішнього сховища використано контейнер `std::vector<T>`. Це забезпечує:

- ефективний доступ до елементів за індексом
- безперервне розміщення даних у пам’яті
- добре кешування при ітерації

Для реалізації пошуку використано алгоритми стандартної бібліотеки з простору імен `std::ranges`. Це відповідає сучасному стилю C++20 і вище.

- Метод `find_first` використовує алгоритм `std::ranges::find`, який виконує пошук першого елемента, що дорівнює заданому значенню. Для цього тип `T` має підтримувати операцію порівняння на рівність.

- Метод `find_first_if` використовує алгоритм `std::ranges::find_if`. У якості параметра передається предикат, який визначає умову пошуку. Це дозволяє виконувати пошук за довільними критеріями.
- Метод `find_all_if` реалізує пошук усіх елементів, що задовольняють умові. Для цього використовується ітерація по контейнеру з перевіркою предиката та накопиченням результатів у новому контейнері.

У якості предикатів використовуються лямбда вирази. Це дозволяє задавати умови без створення окремих функцій або функціональних об'єктів.

Клас надає методи `begin` та `end`, які повертають ітератори до внутрішнього контейнера. Це дозволяє використовувати об'єкт класу як джерело даних для стандартних алгоритмів, для прикладу `std::for_each` був застосован у тесті. `std::for_each` це класичний алгоритм стандартної бібліотеки з `<algorithm>`. Він працює через пару ітераторів:

```
std::for_each(begin, end, func);
```

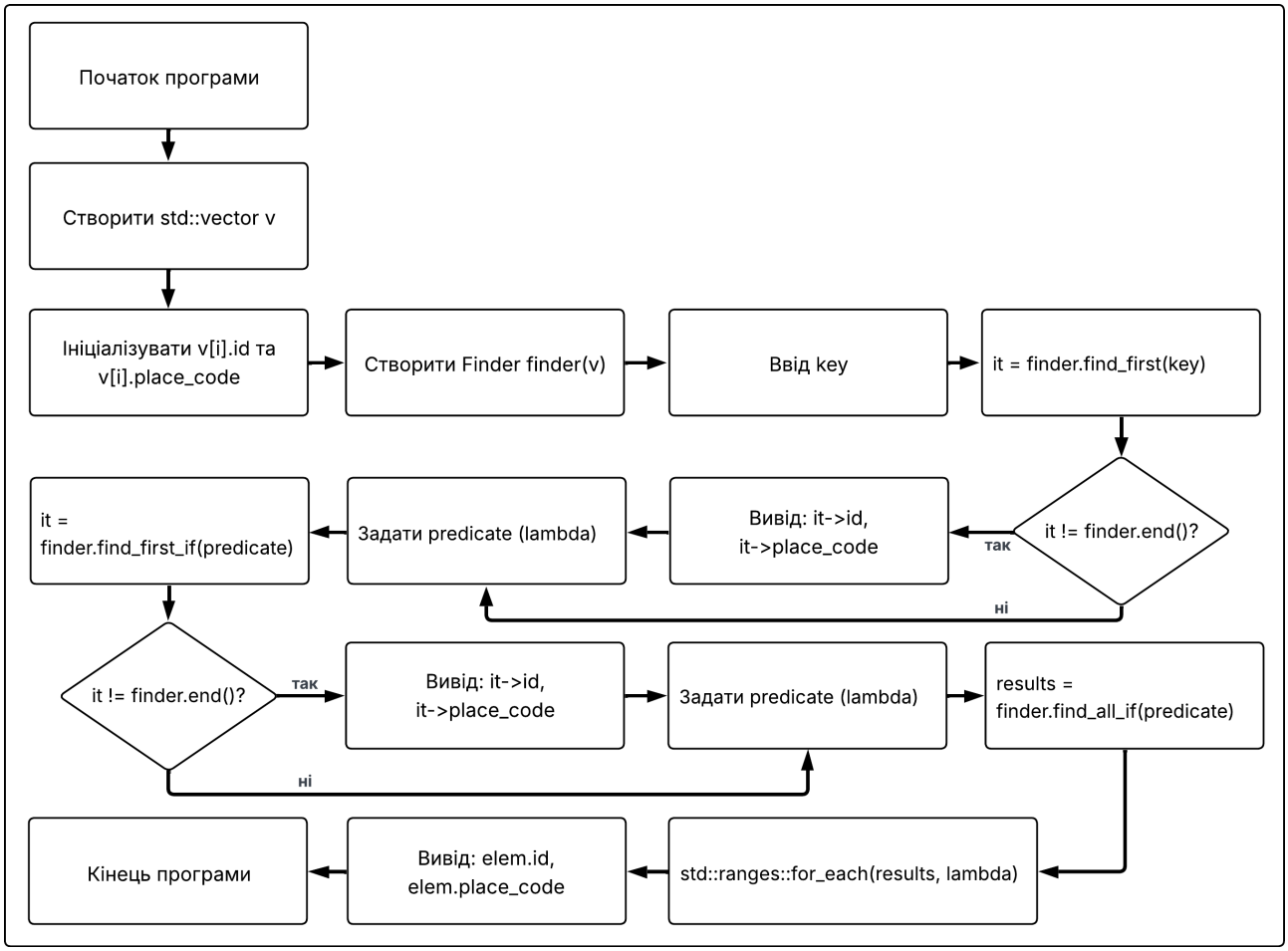

Рис. 1 Blank diagram.png

```
/home/tysser/Scripts/jetbrains/cpp_course/cmake-build-debug/I
Знайдено точний збіг: { id: 3 place_code: B202 }

Перший з place_code A101: { id: 2 place_code: A101 }

Усі з place_code A101:

{ id: 2 place_code: A101 }
{ id: 4 place_code: A101 }
{ id: 6 place_code: A101 }

Process finished with exit code 0
```

Рис. 2 Результат виконання

1.4 Висновок

Реалізовано узагальнений шаблонний клас для пошуку елементів у контейнері. Продемонстровано застосування сучасних можливостей мови - `std::ranges`, лямбда вирази та узагальнені алгоритми. Показано, як алгоритми, відокремлено від структур даних, взаємодіють з ними через ітератори.